

Project Proposal: Multimodal Math: Translating Visual Diagrams into Lean

1. Motivation and Background

Mathematics is not just symbolic, it is also very visual. When people solve geometry problems, they usually rely on diagrams to understand relationships like intersections, midpoints, or parallel lines. However, in formal systems like Lean 4, everything must be written explicitly in code. This creates a gap between how humans think about math (visually) and how computers verify math (symbolically).

Recently, large language models (LLMs) have made progress in translating text into formal proofs. But most of this work is focused on text-based problems. In practice, many mathematical problems, especially in geometry, depend heavily on diagrams. Current AI systems struggle to correctly interpret diagrams and convert them into formal logic.

One key issue is that vision-language models (VLMs) often make incorrect assumptions. For example, if a triangle looks symmetric, the model might assume it is isosceles even when that is not given. These small mistakes make the generated Lean code invalid or impossible to prove.

This project focuses on this exact gap:

How can we reliably convert visual mathematical information into correct formal representations?

Instead of directly trusting the model, we aim to introduce a structured pipeline that extracts multiple possible interpretations and filters them before using them in Lean.

2. Proposed Deliverable

The main deliverable of this project is a **Python-based pipeline** that connects visual input to Lean 4.

The system will have three main components:

1. Diagram Understanding (VLM Layer)

A vision-language model will take an image (e.g., a geometry diagram) and identify objects like points, lines, and circles, along with relationships such as “point D lies on BC” or “two lines intersect.”

2. Candidate Generation

Instead of generating a single interpretation, the system will produce multiple possible formal descriptions (hypotheses). This helps reduce errors from incorrect visual assumptions.

3. Validation + Lean Output

A filtering step will check whether the generated statements are logically consistent. The final output will be a Lean 4 file that defines the objects and theorem statement.

Example output (simplified):

```
import LeanGeo.Euclidean

variable (A B C D : Point)

/--
In triangle ABC, D is the midpoint of BC.
--/
theorem example (h1 : IsTriangle A B C) (h2 : D = midpoint B C) :
  [goal] :=
by
  sorry
```

The goal is not to fully solve proofs, but to **correctly initialize them**, which is currently the hardest step.

3. First Steps and Initial Findings

We have already started exploring how current VLMs handle simple diagrams.

Initial Observations:

- Models are good at detecting objects (e.g., “triangle ABC”)
- But they struggle with **implicit constraints** (e.g., assuming symmetry or equal lengths without justification)

This confirms that directly trusting the model is not reliable.

Planned Approach:

- Start with **simple geometry diagrams** (triangles, midpoints, intersections)
- Use structured prompts to force the model to output:
 - Objects (points, lines)
 - Relationships (collinearity, intersection)
- Convert this into a structured format before generating Lean code

Early Direction:

We will focus on **geometry only**, since it is visual and well-defined. This keeps the project manageable and aligned with existing Lean geometry libraries.

4. Resources and Tools

We plan to use the following tools:

Models:

- Vision-language models (e.g., GPT-4o, Gemini) for diagram understanding

Datasets:

- Geometry-focused datasets (simple diagrams first)
- Possibly benchmark datasets with diagram-based math problems

Formal Systems:

- Lean 4
- LeanGeo or mathlib4 (for geometry definitions)

Programming:

- Python (for building the pipeline and connecting components)

5. Intermediate Milestones and Timeline

Week 1: Setup + Data

- Choose dataset or create simple diagrams
- Set up Lean 4 environment
- Test basic VLM outputs on diagrams

Week 2: Structured Extraction

- Design prompts to extract objects and relationships
- Convert outputs into structured format (JSON-like)

Week 3: Lean Generation

- Build script to convert structured output into Lean code

- Generate valid theorem headers

Week 4: Validation + Testing

- Add simple filtering (remove inconsistent assumptions)
- Test on multiple examples
- Analyze common failure cases

Final Deliverable:

- Working pipeline (image $\rightarrow$ Lean code)
- Short report explaining results and limitations

6. Conclusion

This project aims to bridge the gap between visual reasoning and formal verification. While current systems are good at symbolic reasoning, they struggle with interpreting diagrams correctly.

By building a pipeline that combines vision models with structured validation, we hope to show that:

- Visual understanding can be integrated into formal systems like Lean
- The biggest challenge is not solving proofs, but **correctly representing the problem**

Unlike prior work that assumes a single correct interpretation, we model ambiguity and validate multiple candidate formalizations.

If successful, this approach could make formal verification more accessible and closer to how humans actually think about mathematics.

Aaditya Baruah (abb2237)
Sahasra Kokkula (sk5652)